# Worksheet — 1.2 Software & Software Development — ANSWER SHEET

*Separate answer sheet / mark scheme — keep for marking.*

## Answer key

**1. Key-term match:** 1 → C; 2 → E; 3 → A; 4 → F; 5 → D; 6 → B.

**2. Stages of compilation:** - Stage 1 — **Lexical** analysis: code → **tokens**; remove whitespace/comments; build **symbol** table. - Stage 2 — **Syntax** analysis (also called **parsing**): tokens checked vs **grammar**; **parse** tree built; **syntax** errors reported here. - Stage 3 — Code **generation**: produces **object** / machine code from the parse tree + symbol table. - Stage 4 — Code **optimisation**: refines code to run **faster** / use less **memory**, same result.

**3. Translators:**

| | **Compiler** | **Interpreter** |
|---|---|---|
| When | All at once, before the program runs | Line by line, at run time |
| Output | Standalone executable (.exe) | None |
| Program speed | Fast (already machine code) | Slower (translated each run) |
| Source needed to run? | No (executable only) | Yes, plus the interpreter |
| Errors | All reported at the end | Stops at the first error found |
| Best for | Distributing finished software | Developing/debugging, scripting |

Assembler: translates **assembly language** into machine code, at roughly **one mnemonic : one machine instruction** (one-to-one).

**4. Methodology match:** a) **Spiral**; b) **Waterfall**; c) **RAD**; d) **Extreme Programming (XP)**; e) **Agile**.

**5. Addressing modes (worked):** For `LDA 20` with [20]=35, [35]=99, [99]=12: - a) **Immediate** — operand *is* the value → loads **20**. - b) **Direct** — operand is the address holding the value → go to address 20, find 35 → loads **35**. - c) **Indirect** — operand is the address of a location holding the *address* of the value → address 20 holds 35; address 35 holds 99 → loads **99**. - d) **Indexed** — operand = base address + contents of the **index register**; used to step through **arrays**.

**6. Scheduling & interrupts:** - a) **Round robin** — it **is** pre-emptive (the time slice forces a switch). - b) Any one: requires each job's total runtime to be known/estimated in advance; **long jobs can starve** if short jobs keep arriving; not pre-emptive once a job starts. - c) **Pre-emptive** = a running process can be stopped (interrupted) and the CPU reallocated to another process before the first finishes. - d) Order: CPU checks interrupt register (1) → finish current instruction if higher priority (2) → push registers onto stack (3) → load address of correct ISR (4) → run ISR (5) → pop registers off stack and resume (6).

| Step | Order |
|---|---|
| Load the address of the correct ISR | 4 |
| Pop the registers off the stack and resume the original task | 6 |
| CPU checks the interrupt register at the end of the F-D-E cycle | 1 |
| Run the ISR | 5 |
| Push the registers (e.g. PC, ACC) onto the stack | 3 |
| Finish the current instruction (if the interrupt is higher priority) | 2 |

- e) A stack is **LIFO**, so when a higher-priority interrupt occurs while another ISR is running (**nested interrupts**), the registers are restored in the correct reverse order and every task resumes exactly where it left off.

**7. OOP match:** instance → **Object**; template/blueprint → **Class**; variable/data → **Attribute**; subroutine/behaviour → **Method**; bundling + data hiding → **Encapsulation**; subclass acquiring superclass features → **Inheritance**; same method name, different behaviour → **Polymorphism**.

**8. Challenge:** - (i) **Compiler.** Reasons (any two): it produces a **standalone executable** so customers do **not** receive the source code; the program runs **fast** because it is already machine code; the user does **not** need the source or an interpreter installed to run it. - (ii) Virtual memory uses **secondary storage (a swap/page file)** *as if* it were RAM — the physical RAM is unchanged, and disk access is much **slower** than real RAM. Excessive swapping of pages between RAM and disk causes **disk thrashing**, where the machine spends most of its time paging and grinds to a halt.

**9. Interrupt handling with a nested interrupt — order** 1 — At the end of the current FDE cycle, the CPU checks the interrupt register and finds a flag set. 2 — The printer interrupt's priority is compared with the current task's; it is higher, so it will be serviced. 3 — The contents of the registers (PC, ACC, etc.) are pushed onto the stack. 4 — The PC is loaded with the address of the printer's ISR (from the interrupt vector), and the ISR begins to run. 5 — The power-failure interrupt occurs: the printer ISR's register values are pushed onto the stack, and the higher-priority ISR runs to completion first. 6 — The printer ISR's registers are popped off the stack and the printer ISR finishes running. 7 — The registers are popped off the stack in reverse order and the word processor resumes exactly where it left off. *(Key ideas: check flag at the end of a cycle → compare priority → save state on the stack → vector to the ISR; the nested, higher-priority interrupt is fully serviced before the printer ISR resumes; the LIFO stack guarantees restores happen in reverse order.)*

**10. Which methodology — and why?** (justification must quote the scenario; example answers)

| Scenario | Methodology | Justification |
|---|---|---|
| a) Games demo, sprints, tester feedback | **Agile** | "A playable demo within weeks… evolve it sprint by sprint using tester feedback" describes iterative working increments refined by regular user feedback. |
| b) Tax software, fixed legal rules, sign-off | **Waterfall** | The requirements are "fully specified" and "legally fixed", and "formal documentation and sign-off at each stage" matches waterfall's linear, document-driven stages. |
| c) Defence project, risk per phase, prototype per loop | **Spiral** | "Evaluate the risks of each phase, building a prototype before more budget is committed" is exactly spiral's four-quadrant, risk-driven iteration. |
| d) Heart-rate firmware, pair-quality, on-site customer, tiny releases | **Extreme Programming (XP)** | Critical quality with "a customer representative available daily" and "small, constantly-tested increments" matches XP's pair programming, on-site customer and test-driven small releases. |
| e) Urgent internal tool, throwaway prototypes, UI over efficiency | **RAD** | "Needed urgently" with "users refining throwaway prototypes" and usability prioritised over efficient code is characteristic of rapid application development. |

## Section B — model answers

**Q1 (2 marks, AO1)** *Model answer:* An operating system manages the computer's memory, deciding which programs and data are loaded into which parts of RAM (e.g. by paging), and it schedules the processor, deciding which process gets CPU time and for how long. *Mark points:* any two of: memory management (paging/segmentation/virtual memory) **[1]**; processor/process scheduling **[1]**; management of input/output devices (via device drivers) **[1]**; file management **[1]**; security (user accounts/access rights) **[1]**; providing a user interface **[1]**; handling interrupts **[1]**. Max 2.

**Q2 (2 marks, AO1)** *Model answer:* During lexical analysis the source code is broken up into tokens — the keywords, identifiers, operators and constants of the language — while whitespace and comments are removed. At the same time a symbol table is created, storing details (such as names and data types) of the variables and subroutines found. *Mark points:* any two of: source code converted into tokens **[1]**; whitespace/comments removed **[1]**; symbol table created holding identifiers and their details **[1]**. Max 2.

**Q3 (3 marks, AO2)** *Model answer:* With an interpreter each line is translated and executed at run time, so the student can run her code immediately after every small change without waiting for a full compile — ideal when she re-runs many times each lesson. When there is a mistake, the interpreter stops at the first error and reports its location, helping her find and fix bugs one at a time as she learns. She keeps and works directly from the source code throughout, which does not matter here because she is not distributing the program to anyone. *Mark points:* translation happens line by line at run time, so changes can be tested instantly / no separate compile step **[1]**; execution stops at the first error, making bugs easy to locate — suits debugging while learning **[1]**; application to scenario — she makes small, frequent changes and re-runs constantly / speed of the finished program and protecting source code are irrelevant to her **[1]**.

**Q4 (4 marks, AO2)** *Model answer:* In the waterfall lifecycle, all requirements must be fixed at the start, and each stage must be completed before the next begins; the client only sees the finished product at the end. This suits the charity badly because the trustees do not yet know exactly what they want, and changing requirements later in waterfall means going back up through the stages, which is slow and expensive. Agile, by contrast, welcomes changing requirements: the site is built in short iterations, each producing a working version. The trustees can try out these working increments regularly, feed back, and

have their new ideas incorporated into the next iteration, so the final website is far more likely to be what they actually need. *Mark points:* waterfall needs requirements fixed up-front / changes are costly because stages must be revisited **[1]**; applied: the trustees' requirements are vague and expected to change **[1]**; agile is iterative and accommodates change, producing working versions each iteration **[1]**; applied: the trustees want to try working versions regularly and feed back, shaping later iterations **[1]**.

**Q5 (4 marks, AO2)**

| Addressing mode | Value loaded into ACC |
|---|---|
| Immediate | 40 |
| Direct | 55 |
| Indirect | 88 |
| Indexed | 21 |

*Mark points:* one mark per row **[4]**. Working: immediate — the operand itself is the data → **40**; direct — the operand is the address of the data → contents of address 40 → **55**; indirect — the operand is the address of the address of the data → address 40 holds 55, address 55 holds **88**; indexed — effective address = operand + index register = 40 + 3 = 43 → contents of address 43 → **21**.

**Q6 (2 marks, AO1)** *Model answer:* An interrupt is a signal sent to the processor, by hardware or software, indicating that a device or process needs its attention, causing the current task to be suspended so the interrupt can be serviced. For example, a printer signals that it has run out of paper. *Mark points:* signal to the CPU (from hardware or software) requesting attention / causing the current task to be suspended so an ISR can run **[1]**; valid example, e.g. printer out of paper, key pressed / mouse clicked, I/O operation complete, timer (end of time slice), power failure, division by zero **[1]**.

**Q7 (4 marks, AO1)** *Model answer:* Paging divides memory into fixed-size physical blocks (pages, typically all the same size, mapped onto frames in RAM), whereas segmentation divides a program into variable-sized logical blocks. Also, page boundaries are arbitrary — a page is simply the next fixed-size chunk regardless of the program's structure — whereas segments follow the logical structure of the program, so a whole subroutine, module or block of data forms one segment. *Mark points:* pages are fixed size **[1]**; segments are variable size **[1]**; pages are physical divisions, made without regard to program structure **[1]**; segments are logical divisions that reflect the program's structure (e.g. a complete subroutine/module) **[1]**. (Both allow a program to be held in memory non-contiguously and both are used to implement virtual memory — creditworthy context but the marks are for the differences.)

**Q8* (9 marks, AO3) — levels of response**

*Level descriptors:* - **Level 3 (7–9 marks):** Thorough knowledge and understanding of both methodologies. The discussion is consistently **applied to the drone scenario** (safety-critical core vs evolving customer-facing requirements), weighs strengths and weaknesses of **both** approaches, and presents a **well-developed, balanced line of reasoning** leading to a supported recommendation. - **Level 2 (4–6 marks):** Reasonable knowledge and understanding; some application to the scenario, but the discussion may favour one methodology without weighing the other, or the reasoning is only partly developed; any recommendation has limited support. - **Level 1 (1–3 marks):** Basic, generic descriptions of waterfall/agile with little or no application to the scenario; points made in isolation with no coherent line of reasoning; no meaningful recommendation. - **0 marks:** No creditworthy response.

*Model Level-3 answer:* The waterfall lifecycle works through fixed stages — analysis, design, implementation, testing, maintenance — completing and documenting each before the next begins. For

the safety-critical flight-control core of the drone software this has real strengths: requirements such as "the drone must land safely if a motor fails" can be fully specified up front, and the thorough documentation and formal sign-off at each stage provide the audit trail that regulators of a system flying over the public are likely to demand. Its weakness is rigidity: because testing comes late and stages are not revisited cheaply, a change in requirements mid-project is slow and expensive, and the client has already admitted that some requirements will change.

Agile methodologies build the system in short iterations, delivering working software regularly and welcoming changing requirements. This suits the customer-tracking features well: as the client's ideas evolve, each iteration can incorporate the feedback, and the client sees working increments rather than waiting until the end. However, agile's lighter documentation and its tolerance for redesign in flight are a poor match for safety-critical code, where a "fail fast and fix it next sprint" culture is unacceptable — a fault could crash a drone in a public place — and where evidence of rigorous verification matters more than speed of change.

Neither methodology alone fits the whole project: pure waterfall would handle the safety case but fail the evolving tracking requirements; pure agile would delight the client on features but weaken the assurance the flight software needs.

**Recommendation:** the company should split the project. The safety-critical flight-control software should be developed with a waterfall (or similarly plan-driven) process, with fixed requirements, full documentation and exhaustive staged testing; the customer-facing tracking features, which are still evolving and are not safety-critical, should be developed with an agile approach in short iterations driven by client feedback. This applies each methodology where its strengths matter and its weaknesses do least harm.

*Indicative content (AO1):* waterfall = linear, documented stages, requirements fixed early, testing late; agile = iterative, working increments, embraces changing requirements, less documentation. *(AO2 application):* safety-critical core suits up-front specification, rigorous testing, audit trail; tracking features are evolving so suit iteration and feedback. *(AO3 evaluation):* weighs both methodologies against both halves of the scenario; recognises the tension; reasoned recommendation (e.g. hybrid/split approach, or a justified single choice with mitigations).